Լաբորատոր աշխատանք 9

Դասի դինամիկ անդամներով խնդիրներ՝ մաթեմատիկական անալիզի թեմաներով

Աշխատանքի նպատակը. Ուսումնասիրել C++ լեզվում դինամիկ զանգվածների և դասերի կիրառումը, իրականացնել պարամետրերով և պատճենող կոնստրուկտորներ, վերագրման օպերատոր և դեստրուկտոր։ Էյլերի թվային մեթոդի միջոցով լուծել Բեռնուլիի դիֆերենցիալ հավասարումը, հաշվել ֆունկցիայի արժեքները և որոշել ֆունկցիայի նվազագույն արժեքը։

Խնդրի պահանջները.

● Յուրաքանչյուր խնդրի համար ստեղծել առանձին դաս՝ խնդրին համապատասխան անվանումով։ Դասի private դաշտերում նախատեսել double* դինամիկ զանգվածներ և դրանց չափը։

● Իրականացնել պարամետրերով կոնստրուկտոր, պատճենող կոնստրուկտոր, վերագրման օպերատոր և դեստրուկտոր (Rule of Three)։ Պատճենումը պետք է լինի խորքային։

● Հաշվարկները կատարել առանձին մեթոդով, արդյունքները պահել դինամիկ զանգվածներում, ցուցադրել աղյուսակով և հաշվել խնդրում պահանջված ամփոփիչ ցուցանիշը։

● Սովորական y′ = f(x,y) Կոշիի խնդիրների համար օգտագործել Էյլերի մեթոդը՝ y[i+1] = y[i] + h·f(x[i],y[i])։ Ընդունել h = 0.05, N = 20, եթե այլ բան նշված չէ։

● Լոգարիթմների համար ստուգել սահմանման տիրույթը։ Համասեռ հավասարումների դեպքում կիրառել y = vx փոխարինումը՝ v′ = (f(1,v) – v)/x։ Բեռնուլիի հավասարումների համար կիրառել z = y^(1–n) փոխարինումը։

● Լագրանժի հավասարումների համար օգտագործել տրված պարամետրական լուծումները՝ p արժեքների դինամիկ զանգվածից ստանալով x և y արժեքները։ Սա խուսափում է անորոշ ածանցյալի ուղղակի թվային որոնումից։

● Ստուգել երկու անկախ օբյեկտ և դրանց խորքային պատճենումը։ Մուտքային սխալ արժեքների դեպքում արտածել հասկանալի հաղորդագրություն։

Առաջադրանք. Բեռնուլիի հավասարում Տրված է. y′ + y = x·y², y(0) = 1. Պահանջվում է. Կիրառել z = 1/y փոխարինումը, թվային եղանակով գտնել z-ն, վերականգնել y-ն և որոշել նվազագույն y արժեքը։

```
#include <iostream>
#include <iomanip>
using namespace std;

class BernoulliEquation {
private:
    double* x, * z, * y;
    int size;
    double h;
public:
    BernoulliEquation(int n, double step) {
        size = n + 1;
        h = step;
        x = new double[size];
        z = new double[size];
        y = new double[size];
    }
    BernoulliEquation(const BernoulliEquation& other) {
        size = other.size;
        h = other.h;
        x = new double[size];
        z = new double[size];
        y = new double[size];
        for (int i = 0; i < size; i++) {
            x[i] = other.x[i];
            z[i] = other.z[i];
            y[i] = other.y[i];
        }
    }
    BernoulliEquation& operator=(const BernoulliEquation& other) {
```

```
    if (this != &other) {
        double* newX = new double[other.size];
        double* newZ = new double[other.size];
        double* newY = new double[other.size];
        for (int i = 0; i < other.size; i++) {
            newX[i] = other.x[i];
            newZ[i] = other.z[i];
            newY[i] = other.y[i];
        }
        delete[] x;
        delete[] z;
        delete[] y;
        x = newX;
        z = newZ;
        y = newY;
        size = other.size;
        h = other.h;
    }
    return *this;
}
~BernoulliEquation() {
    delete[] x;
    delete[] z;
    delete[] y;
}
void calculate() {
    x[0] = 0;
    z[0] = 1;
    y[0] = 1;
    for (int i = 0; i < size - 1; i++) {
```

```
            x[i + 1] = x[i] + h;
            z[i + 1] = z[i] + h * (z[i] - x[i]);
            y[i + 1] = (z[i + 1] != 0) ? 1.0 / z[i + 1] : 0;
        }
    }
    void display() const {
        cout << fixed << setprecision(4);
        cout << "i\t x\t\t z\t\t y\n";
        for (int i = 0; i < size; i++)
            cout << i << "\t" << x[i] << "\t\t"
            << z[i] << "\t\t" << y[i] << "\n";
    }
    void findMinimum() const {
        int minIndex = -1;
        for (int i = 0; i < size; i++) {
            if (z[i] != 0 &&
                (minIndex == -1 || y[i] < y[minIndex]))
                minIndex = i;
        }
        if (minIndex != -1)
            cout << "Minimum y = " << y[minIndex]
            << ", x = " << x[minIndex] << endl;
    }
};
int main() {
    int N;
    double h;
    cout << "Enter N: ";
    if (!(cin >> N) || N <= 0) {
        cout << "Invalid N!" << endl;
```

```
        return 1;
    }
    cout << "Enter h: ";
    if (!(cin >> h) || h <= 0) {
        cout << "Invalid h!" << endl;
        return 1;
    }
    BernoulliEquation first(N, h);
    first.calculate();
    first.display();
    first.findMinimum();
    BernoulliEquation second(first);
    BernoulliEquation third(2, 0.1);
    third = first;
    return 0;
}
```